

DS	Session	Graph	Question Information	Level 1	Challenge 82
----	---------	-------	----------------------	---------	--------------

Question description

Udaya family is going to order a large pizza and eat it together. A total of n family members will join the order, and there are m possible toppings. The pizza may have any number of toppings.

Each family member gives two wishes concerning the toppings of the pizza. The wishes are of the form "topping x is good/bad". Your task is to choose the toppings so that at least one wish from everybody becomes true (a good topping is included in the pizza or a bad topping is not included).

Input

The first input line has two integers n and m : the number of family members and toppings. The toppings are numbered $1, 2, \dots, m$.

After this, there are n lines describing the wishes. Each line has two wishes of the form "+ x " (topping x is good) or "- x " (topping x is bad).

Output

Print a line with m symbols: for each topping "+" if it is included and "-" if it is not included. You can print any valid solution.

If there are no valid solutions, print "IMPOSSIBLE".

Constraints

- $1 \leq n, m \leq 10^5$
- $1 \leq x \leq m$

▼ Logical Test Cases

Test Case 1

INPUT (STDIN)

3 5
+ 1 + 2
- 1 + 3
+ 4 - 2

EXPECTED OUTPUT

- + + + -

Test Case 2

INPUT (STDIN)

3 5
+ 2 + 2
- 3 + 3
+ 5 - 2

EXPECTED OUTPUT

- + - - +

▼ Mandatory Test Cases

Test Case 1

KEYWORD

void link(int i,int j)

▼ Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

1

Test Case 2

TOKEN COUNT

600

Test Case 3

NLOC

71

Graph2.c +

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 #define MAXV 200005
5
6 int head[MAXV], to[MAXV * 4], nxt[MAXV * 4], edge_cnt;
7 int rhead[MAXV], rto[MAXV * 4], rnxt[MAXV * 4], redge_cnt;
8 int visited[MAXV];
9 int order[MAXV], order_cnt;
10 int scc[MAXV], scc_cnt;
11
12 void link(int i, int j) {
13     to[++edge_cnt] = j;
14     nxt[edge_cnt] = head[i];
15     head[i] = edge_cnt;
16
17     rto[++redge_cnt] = i;
18     rnxt[redge_cnt] = rhead[j];
19     rhead[j] = redge_cnt;
20 }
21
22 void dfs1(int u) {
23     visited[u] = 1;
24     for (int e = head[u]; e; e = nxt[e]) {
25         int v = to[e];
```

```

26     if (!visited[v]) dfs1(v);
27 }
28 order[++order_cnt] = u;
29}
30
31void dfs2(int u, int id) {
32     scc[u] = id;
33     for (int e = rhead[u]; e; e = rnxt[e]) {
34         int v = rto[e];
35         if (!scc[v]) dfs2(v, id);
36     }
37}
38
39int node(int x, int sign) {
40     if (sign == 1) return 2 * x;
41     else return 2 * x + 1;
42}

```

43

```
44int neg_node(int u) {
```

```
45    return u ^ 1;
```

main | CH.SC.U4CSE25225 | Graph2.c

Ln 90, Col 1 Spaces: 4 UTF-8 CRLF C

47

```
48int main() {
```

```
49    int n, m;
```

```
50    if (scanf("%d %d", &n, &m) != 2) return 0;
```

51

```
52    for (int i = 0; i < n; i++) {
```

```
53        char s1, s2;
```

```
54        int u, v;
```

```
55        scanf(" %c %d %c %d", &s1, &u, &s2, &v);
```

```
56        int n1 = node(u, s1 == '+' ? 1 : 0);
```

```
57        int n2 = node(v, s2 == '+' ? 1 : 0);
```

```
58        link(neg_node(n1), n2);
```

```

59     link(neg_node(n2), n1);
60 }
61
62 for (int i = 2; i <= 2 * m + 1; i++) {
63     if (!visited[i]) dfs1(i);
64 }
65
66 for (int i = order_cnt; i >= 1; i--) {
67     int u = order[i];
68     if (!scc[u]) {
69         dfs2(u, ++scc_cnt);
70     }
71 }
72
73 for (int i = 1; i <= m; i++) {
74     if (scc[node(i, 1)] == scc[node(i, 0)]) {

```

```
75     printf("IMPOSSIBLE\n");
76     return 0;
77 }
78 }
79
80 for (int i = 1; i <= m; i++) {
81     if (scc[node(i, 1)] > scc[node(i, 0)]) {
82         printf("+ ");
83     } else {
84         printf("- ");
85     }
86 }
87 printf("\n");
88 return 0;
89}
90
```

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt

3 5

+ 1 + 2

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Graph2.exe

+ + + + +

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt

3 5

+ 2 + 2

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Graph2.exe

+ + + + +

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>